

Hidden Markov Models for Volatility Regime Detection

Under the supervision of *Joseph Rynkiewicz and Jean-Marc Bardet*.

By Marius Requillart and Maxime Delpech.

2026

Abstract

This work studies hidden Markov models as a tool for modelling financial time series subject to regime changes. After presenting the probabilistic framework of HMMs, we detail the main inference and estimation algorithms : Forward for likelihood computation, Viterbi for decoding hidden states, and Baum–Welch for parameter estimation. These methods are first evaluated on simulated data, which makes it possible to compare the estimated regimes and parameters with the true values. They are then applied to the VIX index in order to identify different volatility regimes and analyse their stability as well as the predictive quality of the model. This study thus illustrates the relevance of HMMs for representing and interpreting the latent dynamics of financial markets.

TABLE OF CONTENTS

Table des matières

1	Introduction	3
2	Definition and Properties of Hidden Markov Models	4
2.1	Definition of a Hidden Markov Model	4
2.1.1	Observable and Hidden Random Variables	4
2.1.2	Joint Distributions and Likelihood	4
2.1.3	General Example of a Hidden Markov Model	5
2.1.4	Remarks on the Observations	7
3	Three Main Problems in HMMs	9
3.1	Likelihood Estimation and the Forward Algorithm	9
3.2	Decoding and the Viterbi Algorithm	11
3.3	Estimation and the Baum–Welch Algorithm	15
4	Algorithmic Limitations and Analysis of Estimation Quality	20
4.1	Limitations and Measurement of Likelihood Quality with the Forward Algorithm . .	20
4.1.1	Limitations of the Forward Algorithm	20
4.1.2	Interpretation of the Log-Likelihood	20
4.1.3	Comparison between Models	21
4.1.4	Out-of-Sample Evaluation	21
4.2	Measurement of Viterbi Decoding Quality	22
4.2.1	Difficulty of Evaluation on Real Data	22
4.2.2	Economic Interpretation of the Detected Regimes	22
4.2.3	Comparison with Posterior Decoding	22
4.3	Measurement of the Quality of Baum–Welch Estimates	23
4.3.1	Log-Likelihood Convergence	24
4.3.2	Stability of the Estimated Parameters	24
4.3.3	Economic Interpretation of Regimes	25

4.3.4	Out-of-Sample Validation	25
5	Simulation of an HMM and Application to Financial Data	26
5.1	Simulated Dataset	26
5.2	Application of the Algorithms to Simulated Data	27
5.3	Interpretation of the Results Obtained on the Simulated Data	28
5.3.1	Analysis of the Forward Problem	28
5.3.2	Analysis of the Viterbi Problem	31
5.3.3	Analysis of the Baum–Welch Problem	33
5.4	Detection of Volatility Regimes on the VIX	36
5.5	Analysis of the Quality of the Baum–Welch Estimation on the VIX	39
6	References	42

1 Introduction

Financial time series play a central role in quantitative finance, as they make it possible to analyse the evolution of asset prices, returns, volatility, and trading volumes over time. However, these series often exhibit complex behaviour : they are noisy, irregular, sometimes unstable, and may be characterised by sudden changes in dynamics. A single financial series may thus alternate between relatively calm periods and periods of high uncertainty, which makes its analysis particularly challenging.

Hidden Markov Models constitute a suitable probabilistic framework for modelling this type of phenomenon. Their principle is based on the existence of a latent state, which is not directly observable and evolves over time according to a Markov chain. The available observations, such as financial returns, are then assumed to depend on this hidden state.

The interest of these models lies in their ability to reveal a hidden structure underlying the observed data. They make it possible not only to detect regime changes, but also to provide an economic and financial interpretation of the results obtained. The identified hidden states may, for example, be associated with phases of growth, financial stress, high uncertainty, or a return to a more stable situation. Thus, the analysis is not limited to a statistical classification of the observations : it may also help relate the detected regimes to market events, changes in expectations, or variations in the level of risk.

In this work, we study hidden Markov models and their applications to financial time series. We first present the theoretical framework of these models, before addressing the main inference and estimation algorithms, in particular the Viterbi and Baum–Welch algorithms. We then analyse their use for identifying different market regimes and gaining a better understanding of the evolution of financial series.

2 Definition and Properties of Hidden Markov Models

2.1 Definition of a Hidden Markov Model

Hidden Markov Models (HMMs) are statistical models used in situations where latent states are not directly observable. The observable data are generated by these states according to a conditional probability distribution.

2.1.1 Observable and Hidden Random Variables

Definition. The hidden states are modelled by a Markov process $(X_t)_{t \in \mathbb{N}}$ taking values in a finite set $E = \{1, \dots, N\}$. We denote by $A = (a_{ij})$ the transition matrix defined by :

$$a_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i).$$

Definition. The observations are modelled by a process $(Y_t)_{t \in \mathbb{N}}$ taking values in a set O . Conditionally on the hidden states, the variables (Y_t) are independent and satisfy :

$$\mathbb{P}(Y_t = y \mid X_t = i) = b_i(y),$$

where $B = (b_i(y))$ is the emission matrix (or family).

Definition. A hidden Markov model is defined by the triplet (A, B, π) where :

- A is the transition matrix of the hidden states,
- B is the emission distribution of the observations,
- π is the initial distribution, with $\pi_i = \mathbb{P}(X_1 = i)$.

2.1.2 Joint Distributions and Likelihood

Property : Let $X = (X_1, \dots, X_T)$ be the sequence of hidden states and $Y = (Y_1, \dots, Y_T)$ the sequence of observations. The distribution of the observations conditionally on the sequence of hidden states is :

$$\mathbb{P}(Y \mid X) = \prod_{t=1}^T \mathbb{P}(Y_t \mid X_t)$$

Proof :

$$\mathbb{P}(Y \mid X) = \prod_{t=1}^T \mathbb{P}(Y_t \mid X_1, \dots, X_T, Y_1, \dots, Y_{t-1}) = \prod_{t=1}^T \mathbb{P}(Y_t \mid X_t)$$

Property : Let $X = (X_1, \dots, X_T)$ be the sequence of hidden states and $Y = (Y_1, \dots, Y_T)$ the sequence of observations. The joint distribution of the observations and hidden states is given by :

$$\mathbb{P}(Y, X) = \pi_{X_1} \prod_{t=2}^T \mathbb{P}(X_t | X_{t-1}) \prod_{t=1}^T \mathbb{P}(Y_t | X_t)$$

Proof :

$$\begin{aligned} \mathbb{P}(X, Y) &= \mathbb{P}(X_1, Y_1) \prod_{t=2}^T \mathbb{P}(X_t, Y_t | X_1, \dots, X_{t-1}, Y_1, \dots, Y_{t-2}) \\ &= \mathbb{P}(X_1) \mathbb{P}(Y_1 | X_1) \prod_{t=2}^T \mathbb{P}(X_t | X_1, \dots, X_{t-1}, Y_1, \dots, Y_{t-2}) \mathbb{P}(Y_t | X_1, \dots, X_{t-1}, Y_1, \dots, Y_{t-2}) \\ &= \mathbb{P}(X_1) \mathbb{P}(Y_1 | X_1) \prod_{t=2}^T \mathbb{P}(X_t | X_{t-1}) \mathbb{P}(Y_t | X_t) \\ &= \mathbb{P}(X_1) \prod_{t=2}^T \mathbb{P}(X_t | X_{t-1}) \prod_{t=1}^T \mathbb{P}(Y_t | X_t) \\ &= \pi_{X_1} \prod_{t=2}^T \mathbb{P}(X_t | X_{t-1}) \prod_{t=1}^T \mathbb{P}(Y_t | X_t) \end{aligned}$$

Remark : The distribution of the observations at each time t depends on the hidden state at that time and not on the previous states.

Example : Consider a hidden Markov model with two states A and B . Assuming that the observations are Gaussian conditionally on the hidden state, we have :

$$\mathcal{L}(Y_t | X_t = A) \sim \mathcal{N}(\mu_A, \sigma_A^2)$$

$$\mathcal{L}(Y_t | X_t = B) \sim \mathcal{N}(\mu_B, \sigma_B^2)$$

Remark : X_t acts as a distribution selector for Y_t .

2.1.3 General Example of a Hidden Markov Model

Consider a simple example of a hidden Markov model with two states C and V (for "calm" and "volatile") and Gaussian observations conditional on these states.

Let $E = \{C, V\}$ be the state space, and let π denote the initial distribution, for example $\pi_0 = (0.5, 0.5)$, indicating that the process starts with equal probability in states C and V .

The transition matrix A may be defined as follows :

$$A = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix}$$

with $a_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i)$.

The family of parametric densities B may be defined by the Gaussian distributions conditional on each state :

$$B = \begin{cases} \mathcal{L}(Y_t \mid X_t = C) \sim \mathcal{N}(\mu_C, \sigma_C^2) \\ \mathcal{L}(Y_t \mid X_t = V) \sim \mathcal{N}(\mu_V, \sigma_V^2) \end{cases}$$

Note : The transition matrix A is not known in most applications ; it must be estimated from the observation data, for example using the Baum–Welch algorithm, which we will study later.

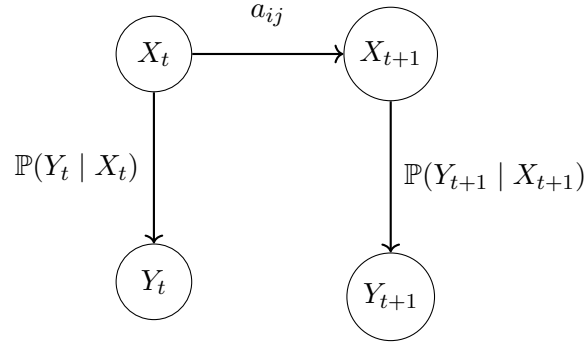


FIGURE 1 – Hidden Markov Model (HMM)

2.1.4 Remarks on the Observations

Property : The total probability of the observations $(Y_t)_t$ is given by :

$$\mathbb{P}(Y) = \sum_X \mathbb{P}(Y, X) = \sum \mathbb{P}(Y | X) \mathbb{P}(X)$$

Property : In a non-degenerate HMM, the observation process $(Y_t)_t$ is not a Markov process, even though the hidden states $(X_t)_t$ are.

Proof :

If the observation process $(Y_t)_t$ were Markovian, then we would have

$$\mathbb{P}(Y_{n+1} | Y_1, \dots, Y_n) = \mathbb{P}(Y_{n+1} | Y_n).$$

However, for K hidden states, we have

$$\begin{aligned} \mathbb{P}(Y_{n+1} | Y_1, \dots, Y_n) &= \sum_{i=1}^K \mathbb{P}(Y_{n+1}, X_{n+1} = i | Y_1, \dots, Y_n) \\ &= \sum_{i=1}^K \mathbb{P}(Y_{n+1} | X_{n+1} = i, Y_1, \dots, Y_n) \mathbb{P}(X_{n+1} = i | Y_1, \dots, Y_n) \\ &= \sum_{i=1}^K \mathbb{P}(Y_{n+1} | X_{n+1} = i) \mathbb{P}(X_{n+1} = i | Y_1, \dots, Y_n). \end{aligned}$$

Moreover,

$$\begin{aligned} \mathbb{P}(X_{n+1} = i | Y_1, \dots, Y_n) &= \sum_{j=1}^K \mathbb{P}(X_{n+1} = i, X_n = j | Y_1, \dots, Y_n) \\ &= \sum_{j=1}^K \mathbb{P}(X_{n+1} = i | X_n = j, Y_1, \dots, Y_n) \mathbb{P}(X_n = j | Y_1, \dots, Y_n) \\ &= \sum_{j=1}^K \mathbb{P}(X_{n+1} = i | X_n = j) \mathbb{P}(X_n = j | Y_1, \dots, Y_n) \\ &= \sum_{j=1}^K a_{ji} \mathbb{P}(X_n = j | Y_1, \dots, Y_n). \end{aligned}$$

and,

$$\begin{aligned}
\mathbb{P}(X_n = j \mid Y_1, \dots, Y_n) &= \mathbb{P}(X_n = j, Y_1 \dots Y_n) / \mathbb{P}(Y_1 \dots Y_n) \\
&= \frac{\mathbb{P}(Y_n \mid X_n = j, Y_1 \dots Y_{n-1}) \mathbb{P}(X_n = j, Y_1 \dots Y_{n-1})}{\mathbb{P}(Y_1 \dots Y_n)} \\
&= \frac{\mathbb{P}(Y_n \mid X_n = j) \mathbb{P}(X_n = j \mid Y_1 \dots Y_{n-1}) \mathbb{P}(Y_1 \dots Y_{n-1})}{\mathbb{P}(Y_n \mid Y_1 \dots Y_{n-1}) \mathbb{P}(Y_1 \dots Y_{n-1})} \\
&= \frac{\mathbb{P}(Y_n \mid X_n = j) \mathbb{P}(X_n = j \mid Y_1 \dots Y_{n-1})}{\mathbb{P}(Y_n \mid Y_1 \dots Y_{n-1})}.
\end{aligned}$$

We therefore have :

$$\mathbb{P}(Y_{n+1} \mid Y_1 \dots Y_n) = \sum_{i=1}^K \sum_{j=1}^K a_{ji} \frac{\mathbb{P}(Y_{n+1} \mid X_{n+1} = i) \mathbb{P}(Y_n \mid X_n = j) \mathbb{P}(X_n = j \mid Y_1 \dots Y_{n-1})}{\mathbb{P}(Y_n \mid Y_1 \dots Y_{n-1})}.$$

The same procedure is used to compute $\mathbb{P}(X_n = j \mid Y_1 \dots Y_{n-1})$, which reduces the dependence to Y_{n-1} . This recursive procedure is continued until the term depending on Y_1 is reached.

Thus, the term

$$\mathbb{P}(X_{n+1} = i \mid Y_1, \dots, Y_n)$$

generally depends on the entire observed past $(Y_1, \dots, Y_n)$, and not only on Y_n .

Therefore, in general,

$$\mathbb{P}(Y_{n+1} \mid Y_1, \dots, Y_n) \neq \mathbb{P}(Y_{n+1} \mid Y_n),$$

which shows that the observation process (Y_t) is not Markovian.

3 Three Main Problems in HMMs

3.1 Likelihood Estimation and the Forward Algorithm

Problem : How can the likelihood $\mathbb{P}(Y)$ of an observation sequence Y under an HMM (A, B, π) be computed efficiently ?

We assume that the transition matrix A , the emission matrix B , and the initial distribution π are known.

We know that the likelihood of Y is given by :

$$\mathbb{P}(Y) = \sum_X \mathbb{P}(Y, X) = \sum_X \mathbb{P}(Y | X) \mathbb{P}(X).$$

However, for N hidden states and an observation sequence of length T , the number of possible hidden sequences is N^T , which makes direct computation exponentially complex if carried out naively. The optimal solution is to use the Forward algorithm, which computes $\mathbb{P}(Y)$ with complexity $O(N^2T)$.

This algorithm is based on dynamic programming and uses an intermediate variable $\alpha_t(i)$ representing the probability of observing the sequence $y_1 \dots y_t$ and being in hidden state i at time t .

Formally, $\alpha_t(i)$ is defined as :

$$\alpha_t(i) = \mathbb{P}(y_1, \dots, y_t, X_t = i).$$

Algorithm 1 Forward Algorithm

Require: Transition matrix $A = (a_{ij})_{1 \leq i, j \leq N}$, emission probabilities $B = (b_i(y))_{1 \leq i \leq N}$, initial distribution $\pi = (\pi_i)_{1 \leq i \leq N}$, observation sequence $Y = (y_1, \dots, y_T)$

Ensure: Likelihood $\mathbb{P}(Y)$

Initialisation

```
1: for  $i = 1, \dots, N$  do  
2:    $\alpha_1(i) \leftarrow \pi_i b_i(y_1)$   
3: end for
```

Recursion

```
4: for  $t = 2, \dots, T$  do  
5:   for  $j = 1, \dots, N$  do  
6:      $\alpha_t(j) \leftarrow b_j(y_t) \sum_{i=1}^N \alpha_{t-1}(i) a_{ij}$   
7:   end for  
8: end for
```

Termination

```
9:  $\mathbb{P}(Y) \leftarrow \sum_{i=1}^N \alpha_T(i)$   
10: return  $\mathbb{P}(Y)$ 
```

where a_{ij} is the transition probability from state i to state j , $b_j(y_t)$ is the probability of observing $Y_t = y_t$ conditionally on being in state j at time t , and π_i is the initial probability of being in state i .

The Forward algorithm thus computes the likelihood of a given observation sequence efficiently, while avoiding the direct computation of all possible hidden-state sequences.

Complexity analysis :

During initialisation, N operations of complexity $O(1)$ are performed to compute $\alpha_1(i)$ for $i = 1, \dots, N$, giving a complexity of $O(N)$.

During recursion, for each time t from 2 to T , N operations are performed to compute $\alpha_t(j)$ for $j = 1, \dots, N$. Each computation of $\alpha_t(j)$ requires a sum of N terms, corresponding to a complexity of $O(N^2)$ at each time t . Thus, the total complexity of the recursion is $O(N^2T)$.

At termination, a sum of N terms is performed to compute $\mathbb{P}(Y)$, corresponding to a complexity of $O(N)$.

Combining these different steps gives a total algorithmic complexity of $O(N^2T)$, which is much more efficient than the exponential complexity $O(N^T)$ of direct computation.

For clarity, the Forward algorithm may be represented in trellis form :

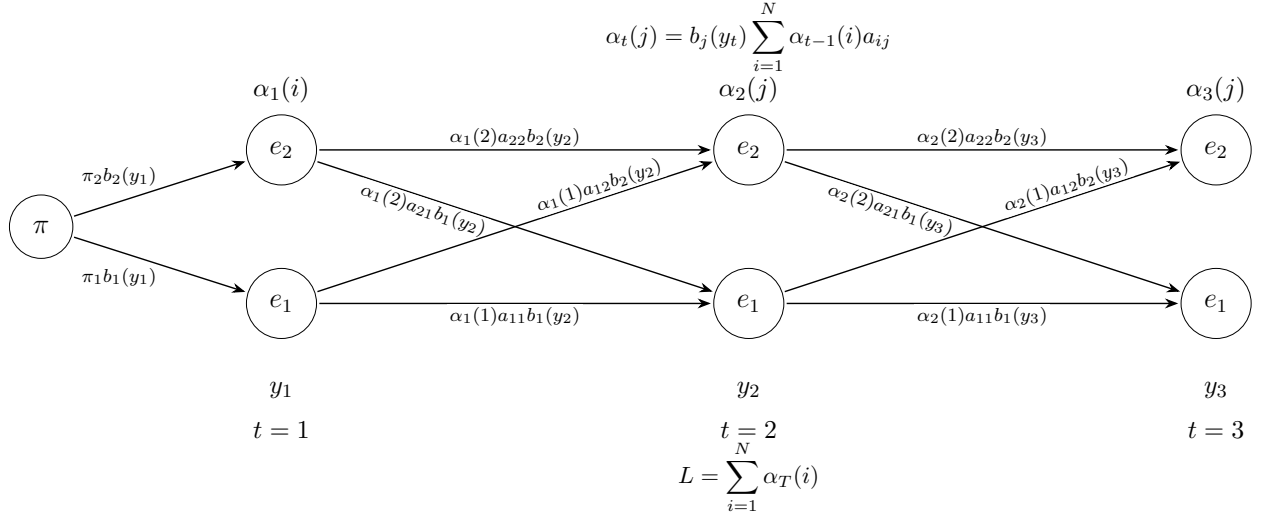


FIGURE 2 – Trellis of the Forward Algorithm

3.2 Decoding and the Viterbi Algorithm

Consider a new problem : decoding a state sequence from a series of observations $Y_1, \dots, Y_T$. We aim to recover the hidden component of the model.

i.e. to recover a posteriori the “true” state sequence $X = (X_1, \dots, X_T)$ that occurred to generate a series of observations, given the initial distribution, transition matrix, and emission probabilities.

It is clear that there is not always a “true” state sequence ; we must therefore introduce optimality criteria to address our problem.

A first approach, which could be described as “greedy”, would be to choose, at each time t , the state X_t for which the probability of observing Y_t is largest. This optimality criterion maximises the expected number of states correctly decoded individually.

However, this may lead to several problems with the resulting state sequence. Since the local criterion ignores the transition matrix, it may propose a transition with zero probability. Nothing in our criterion excludes this ; such a transition could therefore appear in the result even though the resulting sequence would not be feasible.

We must therefore find a criterion that takes into account the probability of occurrence of a state sequence. One could seek to maximise the expected number of correctly recovered pairs of states, or triplets, and so on. However, the most widely used criterion is to find the unique state sequence that maximises the probability of observing our data given the initial distribution, transition matrix, and emission probabilities.

Since the problem has exponential complexity, the solution is to use a dynamic programming algorithm called the Viterbi algorithm.

Viterbi Algorithm First, we note that maximising $\mathbb{P}(X \mid Y, \lambda)$ is equivalent to maximising $\mathbb{P}(X, y \mid \lambda)$, since the observations are fixed.

In order to find the best state sequence $X = \{X_1, \dots, X_T\}$ for the observation sequence $y = \{y_1, \dots, y_T\}$, we define the quantity :

$$\delta_t(i) = \max_{X_1, \dots, X_{t-1}} \mathbb{P}(X_1, X_2, \dots, X_t = i, y_1, y_2, \dots, y_t \mid \lambda)$$

i.e. $\delta_t(i)$ is the probability of the most likely path ending in state i at time t .

With,

$$a_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i)$$

and, if the observations are discrete,

$$b_j(y_t) = \mathbb{P}(Y_t = y_t \mid X_t = j)$$

or, if the observations are continuous,

$$b_j(y_t) = f_{Y_t|X_t=j}(y_t)$$

Thus, recursively, we have :

$$\delta_{t+1}(j) = \left[\max_{1 \leq i \leq N} \delta_t(i) a_{ij} \right] b_j(y_{t+1})$$

Problem : By naively testing all possible hidden-state sequences, there are N^T possible sequences. Moreover, computing the probability of each sequence requires approximately T operations.

Thus, we obtain a complexity of :

$$O(TN^T)$$

This is exponential and therefore unusable as soon as T or N becomes large.

In order to limit this complexity, the Viterbi algorithm relies on the trellis principle.

Trellis Principle :

A column-structured representation in which :

- each column corresponds to a time t
- each row corresponds to a hidden state.
- each node (t, j) represents being in state j at time t .
- each edge between $(t-1, i)$ and (t, j) represents a possible transition, weighted by the probability a_{ij} .

— each node is also associated with the emission probability $b_j(y_t)$.

Thus, the decoding problem amounts to finding the best path through this trellis, from the first column to the last. Each path corresponds to a possible sequence of hidden states. The role of the Viterbi algorithm is precisely to find the path of maximum probability :

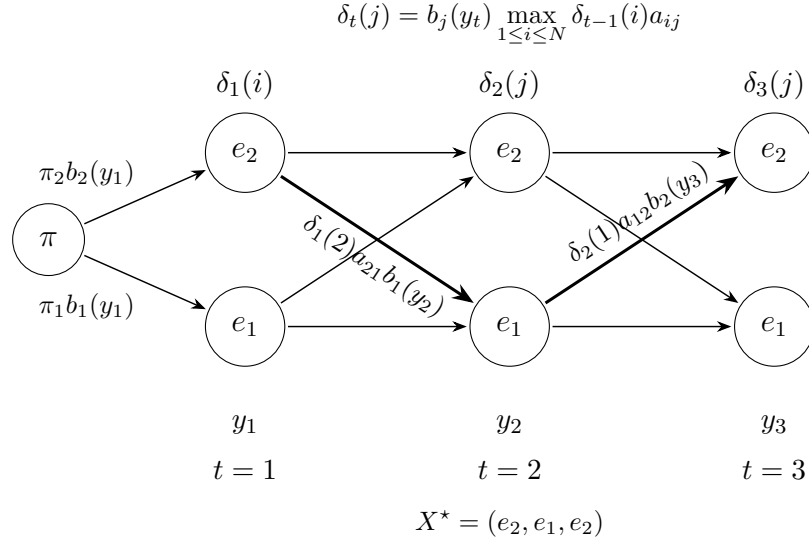


FIGURE 3 – Trellis of the Viterbi Algorithm

The interest of the trellis is that it highlights a fundamental property : in order to recover the state sequence, one must retain the argument that maximises the recurrence. We do this using the list $\psi_t(j)$.

We may therefore use the following algorithm :

Algorithm 2 Viterbi Algorithm

Require: Transition matrix $A = (a_{ij})_{1 \leq i, j \leq N}$, emission probabilities $B = (b_i(y))_{1 \leq i \leq N}$, initial distribution $\pi = (\pi_i)_{1 \leq i \leq N}$, observation sequence $Y = (y_1, \dots, y_T)$

Ensure: Most likely state path $X^* = (X_1^*, \dots, X_T^*)$ and associated probability P^*

Initialisation

```
1: for  $i = 1, \dots, N$  do  
2:    $\delta_1(i) \leftarrow \pi_i b_i(y_1)$   
3:    $\psi_1(i) \leftarrow 0$   
4: end for
```

Recursion

```
5: for  $t = 2, \dots, T$  do  
6:   for  $j = 1, \dots, N$  do  
7:      $\delta_t(j) \leftarrow \left[ \max_{1 \leq i \leq N} \delta_{t-1}(i) a_{ij} \right] b_j(y_t)$   
8:      $\psi_t(j) \leftarrow \arg \max_{1 \leq i \leq N} \delta_{t-1}(i) a_{ij}$   
9:   end for  
10: end for
```

Termination

```
11:  $P^* \leftarrow \max_{1 \leq i \leq N} \delta_T(i)$   
12:  $X_T^* \leftarrow \arg \max_{1 \leq i \leq N} \delta_T(i)$ 
```

Path backtracking

```
13: for  $t = T - 1, T - 2, \dots, 1$  do  
14:    $X_t^* \leftarrow \psi_{t+1}(X_{t+1}^*)$   
15: end for  
16: return  $X^*, P^*$ 
```

We thus obtain the desired state sequence $X_1^*, \dots, X_T^*$.

Complexity analysis :

- At each time, one value is computed for each of the N states.
- For each state, one must evaluate the probability of coming from each of the states.

Thus, at each iteration, the complexity is $O(N^2)$.

There are T time points, giving a total time complexity of :

$$O(TN^2)$$

3.3 Estimation and the Baum–Welch Algorithm

This last problem is the one of greatest interest to us in economic applications. We seek to estimate all parameters of our model, namely the initial distribution, state-transition probabilities, and the parameters of the different observation distributions, using only a series of observations. That is, we seek the model that maximises the probability of observing what has been observed.

i.e.

$$\operatorname{argmax}_{(A,B,\pi)} \mathbb{P}(Y \mid (A, B, \pi))$$

To this end, we may use the algorithm introduced by Baum–Welch, which is in fact a particular application of the Expectation–Maximisation (EM) algorithm.

First, it is important to note that this iterative method makes it possible to approximate a local optimum of our problem.

In order to determine A , B , and π in our problem, we must define the variables with which we will work :

- $\xi_t(i, j) :=$ the probability of being in state i at time t and in state j at time $t + 1$, given the model (A, B, π) and the observation series Y .

$$\forall t \in \{1, \dots, T - 1\}, \quad \forall i, j \in E,$$

$$\begin{aligned} \xi_t(i, j) &= \mathbb{P}(X_t = i, X_{t+1} = j \mid Y, (A, B, \pi)) \\ &= \frac{\mathbb{P}(X_t = i, X_{t+1} = j, Y \mid (A, B, \pi))}{\mathbb{P}(Y \mid A, B, \pi)} \\ &= \frac{\alpha_t(i) a_{ij} b_j(y_{t+1}) \beta_{t+1}(j)}{\mathbb{P}(Y \mid A, B, \pi)} \\ &= \frac{\alpha_t(i) a_{ij} b_j(y_{t+1}) \beta_{t+1}(j)}{\sum_{k \in E} \sum_{\ell \in E} \alpha_t(k) a_{k\ell} b_\ell(y_{t+1}) \beta_{t+1}(\ell)} \end{aligned}$$

- $\gamma_t(i) :=$ the probability of being in state i at time t , given the observation series and the model (A, B, π) .

$$\forall t \in \{1, \dots, T-1\}, \quad \forall i \in E,$$

$$\begin{aligned} \gamma_t(i) &= \mathbb{P}(X_t = i \mid Y, (A, B, \pi)) \\ &= \sum_{j \in E} \mathbb{P}(X_t = i, X_{t+1} = j \mid Y, (A, B, \pi)) \\ &= \sum_{j \in E} \xi_t(i, j) \end{aligned}$$

We note that, by summing $\gamma_t(i)$ over times t , we obtain the number of times state i is visited, or equivalently the expected number of transitions from i .

Similarly, the sum over t of $\xi_t(i, j)$ gives the expected number of transitions from state i to state j .
i.e.

$$\begin{aligned} \sum_{t=1}^T \gamma_t(i) &= \text{expected number of transitions from } i \\ \sum_{t=1}^{T-1} \xi_t(i, j) &= \text{expected number of transitions from } i \text{ to } j \end{aligned}$$

Based on this remark, natural estimators of π , A , and B arise :

$$\forall i, j \in E,$$

$$\pi'_i := \text{probability of being in state } i \text{ at time } 1 = \gamma_1(i)$$

$$a'_{ij} := \frac{\text{expected number of transitions from state } i \text{ to state } j}{\text{expected number of transitions from state } i} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

$$\begin{aligned} b'_j(y) &:= \frac{\text{expected number of observations of } y \text{ while being in state } j}{\text{expected number of times one is in state } j} \\ &= \frac{\sum_{t=1}^T \gamma_t(j) \mathbf{1}_{\{y_t=y\}}}{\sum_{t=1}^T \gamma_t(j)} \end{aligned}$$

When the observations are continuous, the parameters are estimated using natural estimators derived from these quantities.

For example, for Gaussian emissions, we have :

$$\mu_j = \frac{\sum_{t=1}^T \gamma_t(j) y_t}{\sum_{t=1}^T \gamma_t(j)}$$

and

$$\sigma_j^2 = \frac{\sum_{t=1}^T \gamma_t(j) (y_t - \mu_j)^2}{\sum_{t=1}^T \gamma_t(j)}$$

We can therefore derive the following algorithm :

Algorithm 3 Baum–Welch Algorithm

Require: Observation sequence $Y = (y_1, \dots, y_T)$,

- 1: number of states N ,
- 2: initial parameters $\lambda^{(0)} = (A^{(0)}, B^{(0)}, \pi^{(0)})$,
- 3: convergence threshold ε

Ensure: Estimated parameters $\hat{\lambda} = (\hat{A}, \hat{B}, \hat{\pi})$

- 4: Initialise $m \leftarrow 0$

5: **repeat**

E-step : estimation of hidden variables

- 6: For $i = 1, \dots, N$:

$$\alpha_1(i) = \pi_i^{(m)} b_i^{(m)}(y_1)$$

- 7: For $t = 2, \dots, T$ and $j = 1, \dots, N$:

$$\alpha_t(j) = b_j^{(m)}(y_t) \sum_{i=1}^N \alpha_{t-1}(i) a_{ij}^{(m)}$$

- 8: For $i = 1, \dots, N$:

$$\beta_T(i) = 1$$

- 9: For $t = T - 1, \dots, 1$ and $i = 1, \dots, N$:

$$\beta_t(i) = \sum_{j=1}^N a_{ij}^{(m)} b_j^{(m)}(y_{t+1}) \beta_{t+1}(j)$$

- 10: For $t = 1, \dots, T$ and $i = 1, \dots, N$:

$$\gamma_t(i) = \frac{\alpha_t(i) \beta_t(i)}{\sum_{k=1}^N \alpha_t(k) \beta_t(k)}$$

- 11: For $t = 1, \dots, T - 1$ and $i, j = 1, \dots, N$:

$$\xi_t(i, j) = \frac{\alpha_t(i) a_{ij}^{(m)} b_j^{(m)}(y_{t+1}) \beta_{t+1}(j)}{\sum_{k=1}^N \sum_{\ell=1}^N \alpha_t(k) a_{k\ell}^{(m)} b_{\ell}^{(m)}(y_{t+1}) \beta_{t+1}(\ell)}$$

M-step : parameter update

- 12: For $i = 1, \dots, N$:

$$\pi_i^{(m+1)} = \gamma_1(i)$$

- 13: For $i, j = 1, \dots, N$:

$$a_{ij}^{(m+1)} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

- 14: Update the emission parameters $B^{(m+1)}$ using natural estimators based on the $\gamma_t(i)$

- 15: $L^{(m+1)} \leftarrow \sum_{i=1}^N \alpha_T(i)$

- 16: $m \leftarrow m + 1$

- 17: **until** $|L^{(m)} - L^{(m-1)}| < \varepsilon$

- 18: **return** $\hat{\lambda} = \lambda^{(m)}$
-

In this work, we do not address the choice of epsilon, which would require a separate study ; however, it is important to note that this choice may have a significant impact on the convergence of the algorithm. Moreover, with a single observation series, the initial distribution π cannot be robustly estimated, since it only concerns the hidden state at the first time point ; nevertheless, this has little impact on the final use of the model, since the influence of π progressively vanishes over time in favour of the estimated transitions and emissions.

Complexity analysis :

- Forward step : for each time t and each state j , we sum over all previous states. Hence $O(TN^2)$.
- Backward step : as for the Forward step. Hence $O(TN^2)$.
- computation of $\gamma_t(i)$: computation using $\alpha_t(i)$ and $\beta_t(i)$ for each t and each i . Hence $O(TN)$.
- computation of $\xi_t(i, j)$: computation using $\alpha_t(i)$ and $\beta_t(j)$ for each t and each pair (i, j) . Hence $O(TN^2)$.
- update of π_i : the value of $\gamma_1(i)$ is retrieved for each i . Hence $O(N)$.
- update of a_{ij} : computation with a sum over T for each pair (i, j) . Hence $O(TN^2)$.
- update of $b_j(k)$: if one naively iterates over all symbols k , states j , and time points t , then the complexity is $O(TNM)$. However, the observations can be processed efficiently, yielding a complexity of $O(TN)$.

Thus, the overall complexity per iteration is :

$$O(TN^2)$$

If the algorithm performs I iterations before convergence, then the total complexity is :

$$O(ITN^2)$$

4 Algorithmic Limitations and Analysis of Estimation Quality

4.1 Limitations and Measurement of Likelihood Quality with the Forward Algorithm

4.1.1 Limitations of the Forward Algorithm

One of the main limitations of the Forward algorithm arises from the number of observations. Indeed, the longer the observation sequence, the smaller the likelihood $\mathbb{P}(Y)$ becomes, which may lead to computational problems related to numerical precision. To address this problem, the Forward algorithm can be modified by using the log-likelihood. This transforms products into sums, involving larger values that are therefore easier to handle numerically.

Moreover, if the number of hidden states is very large, the Forward algorithm may become extremely computationally expensive, making its use less relevant in applications with a large number of states.

Furthermore, the Forward algorithm evaluates the likelihood of the observations, but it does not directly indicate whether the estimated hidden regimes are economically interpretable. A model may have a good likelihood while producing regimes that are difficult to interpret financially. Thus, the likelihood computed by Forward must be used as an important statistical measure, but it must be supplemented by an analysis of the regimes obtained with Viterbi or with posterior probabilities, as well as by out-of-sample validation.

The Forward algorithm therefore computes the probability of observing a given sequence under a given HMM. If $Y_{1:T}$ denotes the observed series, with T the total number of observations, and if $\lambda = (\pi, A, B)$ denotes the model parameters, the Forward algorithm computes :

$$P(Y_{1:T} \mid \lambda).$$

In practice, the log-likelihood is used instead :

$$\ell(\lambda) = \log P(Y_{1:T} \mid \lambda),$$

because the raw likelihood becomes very small when T is large.

4.1.2 Interpretation of the Log-Likelihood

The log-likelihood measures the model's ability to explain the observations. The higher it is, the greater the probability that the model assigns to the observed series. It therefore constitutes a first measure of the statistical quality of the HMM.

However, this measure must be interpreted with caution. A more complex model, for example with

more hidden states, may obtain a better log-likelihood simply because it has more parameters. A good log-likelihood therefore does not necessarily mean that the estimated regimes are more interpretable, nor that the model will remain effective on new observations.

4.1.3 Comparison between Models

To compare several HMMs, one may use the average log-likelihood per observation :

$$\bar{\ell}(\lambda) = \frac{1}{T} \log P(Y_{1:T} \mid \lambda).$$

This normalisation makes it possible to compare series of different lengths.

Penalised criteria such as AIC and BIC may also be used :

$$AIC = -2\ell(\lambda) + 2p,$$

$$BIC = -2\ell(\lambda) + p \log(T),$$

where p denotes the number of estimated parameters. These criteria penalise models that are too complex. In this case, the best model is the one that minimises AIC or BIC.

4.1.4 Out-of-Sample Evaluation

A good log-likelihood on the training sample is not sufficient. The model may explain past data very well while being less effective on new data. For a financial time series, the dataset is therefore split in chronological order : a first part is used for training, and the following part is used for testing. The likelihood must therefore be evaluated on this test period, which was not used for estimation. If the model $\hat{\lambda}_{\text{train}}$ is estimated on $Y_{1:T_{\text{train}}}$, and if Y_{test} denotes the observations in the test period, one can compute :

$$\ell_{\text{test}} = \log P(Y_{\text{test}} \mid \hat{\lambda}_{\text{train}}).$$

The average test log-likelihood is often used :

$$\bar{\ell}_{\text{test}} = \frac{1}{T_{\text{test}}} \ell_{\text{test}},$$

or the negative log predictive density :

$$NLPD = -\frac{1}{T_{\text{test}}} \ell_{\text{test}}.$$

A low $NLPD$ indicates that the model assigns a high probability to new observations. It is therefore a useful indicator for checking whether the model remains relevant on data that were not used for estimation.

4.2 Measurement of Viterbi Decoding Quality

The Viterbi algorithm makes it possible to estimate the most likely sequence of hidden states given the observations and the model parameters. If $Y_{1:T}$ denotes the observed series, with T the total number of observations, and $\hat{\lambda}$ the estimated model parameters, Viterbi seeks the path

$$\hat{X}_{1:T}^V \in \arg \max_{x_{1:T}} P(X_{1:T} = x_{1:T} \mid Y_{1:T}, \hat{\lambda}).$$

In a financial application, this path may be interpreted as a chronology of market regimes : each date is associated with an estimated regime, for example calm, volatile, stressed, or recovering.

4.2.1 Difficulty of Evaluation on Real Data

The main difficulty is that, with real financial data, the true hidden states are not observed. Hence, no true sequence is available for direct comparison with the estimated sequence. It is then difficult to compute standard quantitative measures such as accuracy, a confusion matrix, or an error rate.

The evaluation of Viterbi is therefore mainly indirect. A very clear Viterbi path is not necessarily the true hidden-state path : it is only the most likely path under the estimated model.

4.2.2 Economic Interpretation of the Detected Regimes

On real data, a first criterion consists in comparing the detected regimes with known financial periods. For example, a regime interpreted as a stress regime should occur more frequently during crises, crashes, or periods of high volatility. Conversely, a calm regime should dominate during more regular periods.

This interpretation remains qualitative, but it is important. If the detected states do not correspond to any clear financial dynamics, the decoding is difficult to use.

The persistence of the regimes may also be examined. A path that changes state at almost every date may indicate that the model is mainly capturing noise. The path must therefore be sufficiently stable to be interpretable.

4.2.3 Comparison with Posterior Decoding

Viterbi does not select the most likely state separately at each date. It selects the globally most likely path, that is, it maximises :

$$P(X_{1:T} = x_{1:T} \mid Y_{1:T}, \hat{\lambda}).$$

This constraint gives a temporally coherent path, but it may lead to the selection, at certain dates, of a state that does not have the highest marginal probability.

Posterior decoding follows a different approach. It computes the posterior probabilities :

$$\hat{\gamma}_t(i) = P(X_t = i \mid Y_{1:T}, \hat{\lambda}),$$

and then chooses, at each date t , the marginally most likely state :

$$\hat{X}_t^P \in \arg \max_{i \in E} \hat{\gamma}_t(i).$$

Thus, posterior decoding makes a local decision date by date, whereas Viterbi optimises a complete trajectory.

Comparing the two approaches is informative, because they do not answer exactly the same question. If Viterbi and posterior decoding give similar classifications, this indicates that the regimes are well identified by the model.

A simple example of a discrepancy measure is the disagreement rate between the two decodings :

$$D_{VP} = \frac{1}{T} \sum_{t=1}^T \mathbf{1}_{\{\hat{X}_t^V \neq \hat{X}_t^P\}}.$$

The closer D_{VP} is to 0, the more similar the regimes provided by the two methods are. Conversely, a high value indicates that the state selection is uncertain or strongly depends on the decoding method. If the two methods give very different results, this reveals substantial uncertainty regarding the hidden states. The Viterbi path must then be interpreted as a likely trajectory, rather than as a certain classification.

4.3 Measurement of the Quality of Baum–Welch Estimates

Baum–Welch estimates the parameters of a hidden Markov model solely from the observations $Y_{1:T}$. If X_t denotes the hidden state at time t , the model is given by :

$$\lambda = (\pi, A, B)$$

with

$$\pi_i = P(X_1 = i), \quad a_{ij} = P(X_{t+1} = j \mid X_t = i)$$

and B is the set of emission distributions $b_i(y)$, or more generally $b_i(y; \theta_i)$ when these distributions depend on parameters θ_i . The algorithm then seeks

$$\hat{\lambda} \in \arg \max_{\lambda} P(Y_{1:T} \mid \lambda).$$

Since this is an EM-type method, the solution obtained is generally a local optimum and not necessarily the best possible model. The quality of the estimates must therefore be studied through

convergence, parameter stability, economic interpretation, and out-of-sample performance.

4.3.1 Log-Likelihood Convergence

At iteration k , let $\ell^{(k)} = \log P(Y_{1:T} \mid \lambda^{(k)})$. Here, T is the length of the observed series, i.e. the total number of observations. Thus, quantities divided by T are average gains per observation.

Baum–Welch must produce a non-decreasing log-likelihood. We therefore monitor :

$$\Delta_k = \ell^{(k)} - \ell^{(k-1)}, \quad \bar{\Delta}_k = \frac{\Delta_k}{T}, \quad r_k = \frac{\Delta_k}{1 + |\ell^{(k-1)}|}.$$

A reasonable stopping criterion is $r_k < 10^{-6}$, or $\bar{\Delta}_k < 10^{-6}$ to 10^{-5} , for several consecutive iterations. These thresholds should not be viewed as absolute rules, but as orders of magnitude for identifying when the improvement becomes negligible.

Convergence that is too rapid may be suspicious if the algorithm stops in fewer than five iterations with parameters close to the initialisation. This case is not necessarily problematic, but it must be compared with the results obtained using other initialisations. When running Baum–Welch M times, one compares :

$$\frac{\ell_{\max}^* - \ell_m^*}{T}, \quad \ell_{\max}^* = \max_{1 \leq m \leq M} \ell_m^*.$$

A difference of 10^{-3} to 10^{-2} per observation may indicate a clearly inferior solution. Conversely, unstable convergence occurs when $\Delta_k < 0$. A small numerical decrease may be tolerated if $\Delta_k/T > -10^{-8}$, but larger or repeated decreases indicate a normalisation, numerical underflow, or implementation problem.

4.3.2 Stability of the Estimated Parameters

Since Baum–Welch depends on the initialisation, the estimation must be repeated with several starting points. Before comparing the results, the states are aligned to correct for label switching : state 1 in one estimate may correspond to state 2 in another. For two estimates (a) and (b), one may measure :

$$D_\pi = \frac{1}{2} \sum_i |\hat{\pi}_i^{(a)} - \hat{\pi}_i^{(b)}|, \quad D_A = \frac{1}{N} \sum_i \frac{1}{2} \sum_j |\hat{a}_{ij}^{(a)} - \hat{a}_{ij}^{(b)}|.$$

The average regime duration is also compared, for $\hat{a}_{ii} < 1$:

$$\hat{d}_i = \frac{1}{1 - \hat{a}_{ii}}, \quad D_{d,i} = \left| \frac{\hat{d}_i^{(a)} - \hat{d}_i^{(b)}}{\hat{d}_i^{(b)}} \right|.$$

Two estimates may have similar likelihoods but regimes with very different persistence.

For discrete emission distributions, one may use :

$$D_B = \frac{1}{N} \sum_i \frac{1}{2} \sum_{y \in \mathcal{Y}} \left| \hat{b}_i^{(a)}(y) - \hat{b}_i^{(b)}(y) \right|.$$

For continuous distributions with densities $\hat{f}_i^{(a)}$ and $\hat{f}_i^{(b)}$, a Kullback–Leibler divergence may be used :

$$\text{KL}(\hat{f}_i^{(a)} || \hat{f}_i^{(b)}) = \int \hat{f}_i^{(a)}(y) \log \left(\frac{\hat{f}_i^{(a)}(y)}{\hat{f}_i^{(b)}(y)} \right) dy.$$

If the emissions are parameterised by θ_i , the vectors $\hat{\theta}_i$ are compared, ideally using a normalised distance to account for scale differences between components.

4.3.3 Economic Interpretation of Regimes

A satisfactory estimate must produce economically interpretable regimes. The emission distributions associated with the states must describe distinct behaviours. If two states have almost identical distributions, the model probably uses too many states or does not clearly separate the regimes. The transition matrix must also be consistent : in financial series, the diagonal coefficients a_{ii} should not be too small, since market regimes often exhibit a certain persistence.

The posterior probabilities

$$\hat{\gamma}_t(i) = P(X_t = i \mid Y_{1:T}, \hat{\lambda})$$

make it possible to verify at which dates each regime dominates. An interpretable regime must correspond to consistent financial periods, for example periods of stress, calm, or high uncertainty.

4.3.4 Out-of-Sample Validation

The terms out-of-sample evaluation, out-of-sample testing, evaluation over a test period, and predictive evaluation on future data are also used. In a financial context, the term backtesting is also used when a method is tested over a past period that was not used for estimation.

The idea is to estimate the model on a training period and then evaluate it on a future period not used for estimation. If $\hat{\lambda}_{train}$ is estimated on $Y_{1:T_{train}}$, one computes :

$$\ell_{test} = \log P(Y_{T_{train}+1:T} \mid \hat{\lambda}_{train}), \quad NLPD = -\frac{1}{T_{test}} \ell_{test}.$$

A low *NLPD* indicates that the model assigns a high probability to new observations. This step is important, since an estimate may explain the past well while generalising poorly to future financial data.

5 Simulation of an HMM and Application to Financial Data

5.1 Simulated Dataset

In order to test and validate the different algorithms studied, we constructed a simulated dataset from a hidden Markov model. The objective of this simulation is to obtain an observation sequence for which the model parameters are known in advance. This then makes it possible to verify whether the inference and estimation algorithms correctly recover the underlying structure of the model.

The model considered has three hidden states, representing three possible weather conditions :

$$S = \{\text{sunny, cloudy, rainy}\}.$$

At each time, the hidden state is not directly observed. However, a variable related to this state is observed, corresponding here to whether or not a person takes an umbrella :

$$O = \{\text{umbrella, no umbrella}\}.$$

The initial state distribution is given by

$$\pi = \begin{pmatrix} 0.5 & 0.3 & 0.2 \end{pmatrix},$$

which means that the simulation starts with probability 0.5 in the “sunny” state, 0.3 in the “cloudy” state, and 0.2 in the “rainy” state.

The transition matrix between the hidden states is defined by :

$$A = \begin{pmatrix} 0.7 & 0.295 & 0.005 \\ 0.3 & 0.4 & 0.3 \\ 0.005 & 0.395 & 0.6 \end{pmatrix}.$$

The emission matrix is defined by :

$$B = \begin{pmatrix} 0.1 & 0.9 \\ 0.4 & 0.6 \\ 0.9 & 0.1 \end{pmatrix}.$$

From these parameters, we simulate a trajectory of length $T = 1000$. The generation is carried out in two steps at each time. First, a hidden state is drawn according to the transition matrix A . Then, an observation is generated according to the emission distribution corresponding to this state, given by the matrix B . We thus obtain two sequences :

$$(X_1, X_2, \dots, X_T)$$

for the hidden states, and

$$(Y_1, Y_2, \dots, Y_T)$$

for the observations.

This simulated dataset provides a framework for evaluating the algorithms. Indeed, unlike a real dataset, the hidden states and true model parameters are known. It is therefore possible to compare the results produced by the algorithms with the values used to generate the data. This will in particular enable us to test the reliability of the different algorithms.

5.2 Application of the Algorithms to Simulated Data

Result of the Forward algorithm : -639.814329

Result of the Viterbi algorithm :

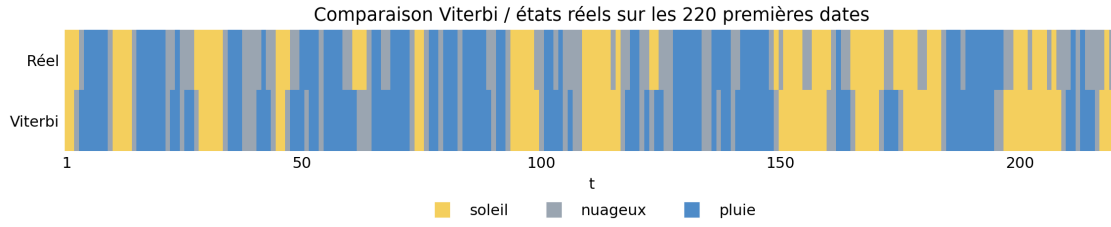


FIGURE 4 – Comparison between the simulated true states and the Viterbi path over the first 220 dates.

Baum–Welch result : After running the Baum–Welch algorithm on the simulated data, we obtain the following estimated parameters.

The estimated transition matrix is

$$\hat{A} = \begin{pmatrix} 0.7026 & 0.2956 & 0.0018 \\ 0.2720 & 0.5963 & 0.1316 \\ 0.1804 & 0.2172 & 0.6024 \end{pmatrix}.$$

The estimated emission matrix is

$$\hat{B} = \begin{pmatrix} 0.1070 & 0.8930 \\ 0.5702 & 0.4298 \\ 0.9895 & 0.0105 \end{pmatrix}.$$

5.3 Interpretation of the Results Obtained on the Simulated Data

5.3.1 Analysis of the Forward Problem

We apply the Forward criteria to the simulated trajectory. The evaluated model is the generating model $\lambda = (\pi, A, B)$.

Likelihood and Log-Likelihood

Criterion	Value
Series length T	1001
Number of free parameters p	11
Raw likelihood	1.3557×10^{-278}
Log-likelihood	-639.814329
Average log-likelihood	-0.639175
NLPD	0.639175
Indicative AIC	1301.628658
Indicative BIC	1355.624961

TABLE 1 – Forward criteria on the complete series.

The raw likelihood is very low, which is expected for a long observation sequence. We therefore use the log-likelihood

$$\ell(\lambda) = \log P(Y_{1:T} \mid \lambda) = -639.814329.$$

The average log-likelihood is

$$\bar{\ell}(\lambda) = \frac{1}{T}\ell(\lambda) = -0.639175,$$

and the negative log predictive density is

$$\text{NLPD} = -\bar{\ell}(\lambda) = 0.639175.$$

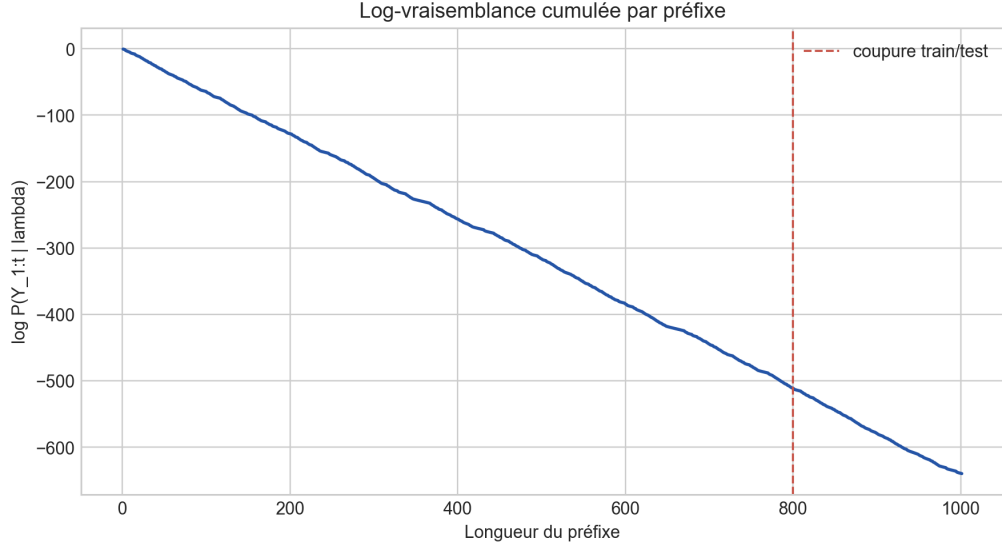


FIGURE 5 – Cumulative log-likelihood by prefix.

Model Comparison

We compare four models of the same dimension :

$$\lambda_0 = (\pi, A, B), \quad \lambda_1 = (\pi, A_{\text{unif}}, B), \quad \lambda_2 = (\pi, A, B_{\text{unif}}), \quad \lambda_3 = (\pi_{\text{unif}}, A_{\text{unif}}, B_{\text{unif}}).$$

The model λ_1 retains the emissions but makes the transitions uniform. The model λ_2 retains the transitions but makes the emissions uniform. The model λ_3 is entirely uniform.

For $N = 3$ states and $M = 2$ observations, we use

$$p = (N - 1) + N(N - 1) + N(M - 1) = 11.$$

The criteria are

$$\text{AIC} = -2\ell(\lambda) + 2p, \quad \text{BIC} = -2\ell(\lambda) + p \log(T).$$

Model	log-lik.	log-lik./obs.	NLPD	AIC	BIC
Simulation model	-639.814329	-0.639175	0.639175	1301.628658	1355.624961
Uniform transitions	-689.395632	-0.688707	0.688707	1400.791264	1454.787566
Uniform emissions	-693.840328	-0.693147	0.693147	1409.680655	1463.676958
Uniform model	-693.840328	-0.693147	0.693147	1409.680655	1463.676958

TABLE 2 – Forward comparison of the models.

The simulation model minimises AIC and BIC. Models with uniform emissions obtain an average

log-likelihood close to $-\log(2)$, which indicates that removing the relationship between hidden states and observations strongly degrades Forward quality.

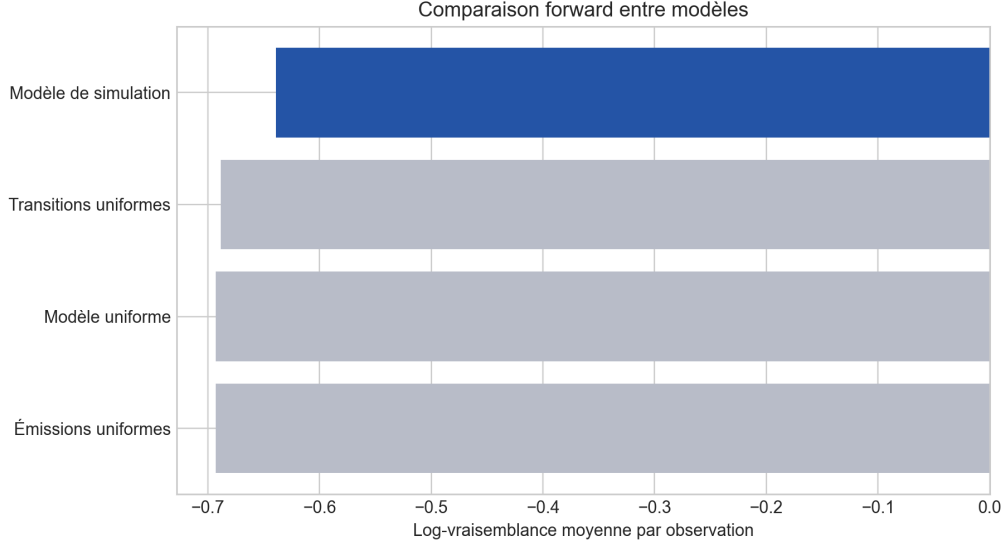


FIGURE 6 – Average log-likelihood by model.

Out-of-Sample Validation

The series is split chronologically into 800 training observations and 201 test observations. Out-of-sample performance is measured by :

$$\ell_{\text{test}} = \log P(Y_{\text{test}} | Y_{\text{train}}, \lambda), \quad \text{NLPD}_{\text{test}} = -\frac{1}{T_{\text{test}}} \ell_{\text{test}}.$$

Segment	T	log-lik.	log-lik./obs.	NLPD
Training	800	-511.319586	-0.639149	0.639149
Conditional test	201	-128.494743	-0.639277	0.639277
Independent test	201	-128.462530	-0.639117	0.639117

TABLE 3 – Out-of-sample Forward validation.

The conditional test NLPD is 0.639277, very close to that obtained on the training part. The model therefore retains stable predictive quality on the test observations.

Conclusion

The Forward criteria provide a consistent evaluation : the average log-likelihood is -0.639175, the NLPD is 0.639175, the generating model minimises AIC and BIC among the models compared, and the out-of-sample NLPD remains close to that obtained over the training period. The comparison

with uniform models shows in particular that the emission matrix plays a central role : when the relationship between hidden states and observations is removed, the likelihood deteriorates substantially.

5.3.2 Analysis of the Viterbi Problem

We apply the criteria related to Viterbi decoding to the simulated trajectory. The path obtained is denoted $\hat{X}_{1:T}^V$.

Interpretation of the Detected Regimes

We first examine whether the decoded states correspond to distinct observation profiles.

Viterbi state	Dates	Observed umbrella	Theoretical umbrella	Observed no umbrella
sunny	408	9.56%	10.00%	90.44%
cloudy	206	12.62%	40.00%	87.38%
rainy	387	100.00%	90.00%	0.00%

TABLE 4 – Observation profile in each state decoded by Viterbi.

The states *sunny* and *rainy* are well separated : the *sunny* state is associated almost always with *No umbrella*, whereas the *rainy* state is associated almost always with *Umbrella*. The *cloudy* state is less clearly identified, which is consistent with less extreme emission probabilities.

We then examine regime persistence.

Sequence	Changes	Rate	Runs	Mean length	Maximum length
Viterbi	326	32.60%	327	3.06	28
Posterior decoding	390	39.00%	391	2.56	21
Local emission decoding	358	35.80%	359	2.79	23

TABLE 5 – Persistence of the decoded sequences.

The Viterbi path makes 326 changes, corresponding to a rate of 32.60%. It is smoother than posterior decoding, which corresponds to its objective : finding a coherent global trajectory.

Comparison with Posterior Decoding and Local Decoding

Posterior decoding chooses, at each date, the marginally most likely state :

$$\hat{X}_t^P \in \arg \max_i P(X_t = i \mid Y_{1:T}, \lambda).$$

Local emission decoding chooses, at each date, the state that makes the current observation most likely :

$$\hat{X}_t^E = \arg \max_i b_i(Y_t).$$

We compare the three decodings using the disagreement rate

$$D_{VP} = \frac{1}{T} \sum_{t=1}^T \mathbf{1}_{\{\hat{X}_t^V \neq \hat{X}_t^P\}}.$$

Criterion	Value
D_{VP}	8.39%
Viterbi vs local emission disagreement	24.48%
Mean posterior probability of the Viterbi state	66.04%
Median posterior probability of the Viterbi state	67.99%
Mean posterior margin	38.33%
Median posterior margin	42.12%

TABLE 6 – Comparison between Viterbi and posterior decoding.

We obtain $D_{VP} = 8.39\%$. The two methods therefore yield similar classifications. The mean posterior probability of the state selected by Viterbi is 66.04%, indicating that the Viterbi path most often follows marginally plausible states.

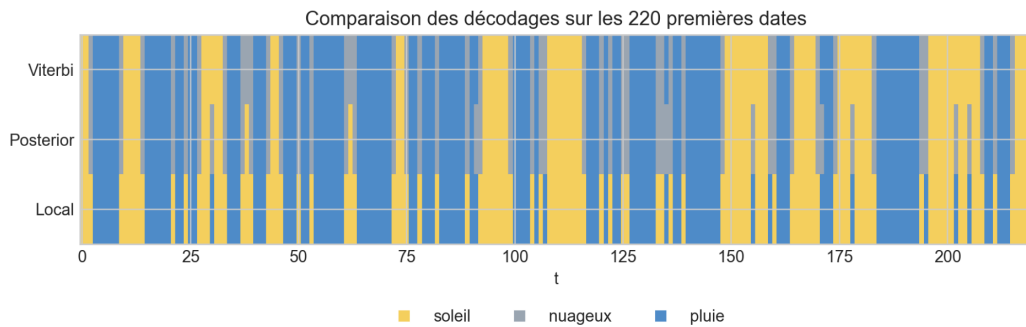


FIGURE 7 – Visual comparison between Viterbi, posterior decoding, and local decoding.

Conclusion

The criteria provide a consistent interpretation of the decoding. The extreme regimes are readily interpretable from the observations, and the disagreement rate with posterior decoding remains low. The Viterbi path is also smoother than decodings based on local decisions. The *cloudy* regime remains less clearly separated, which is consistent with less discriminating emission probabilities.

5.3.3 Analysis of the Baum–Welch Problem

We applied the Baum–Welch algorithm to the simulated dataset presented previously, running the estimation from several different initialisations. The objective here is to observe whether the estimated parameters are stable from one initialisation to another and whether the solutions obtained lead to comparable likelihoods.

Figure 8 compares the final log-likelihoods obtained for the different initialisations. The differences in log-likelihood per observation are very small, of order 10^{-4} to 10^{-3} . This means that, from a likelihood perspective, the different estimates explain the observations in a rather similar manner. Some initialisations, in particular initialisations 9 and 10, give results that are almost identical to the best solution. Conversely, initialisation 8 has the largest difference, indicating a slightly inferior solution, although the difference remains small once scaled by the number of observations.

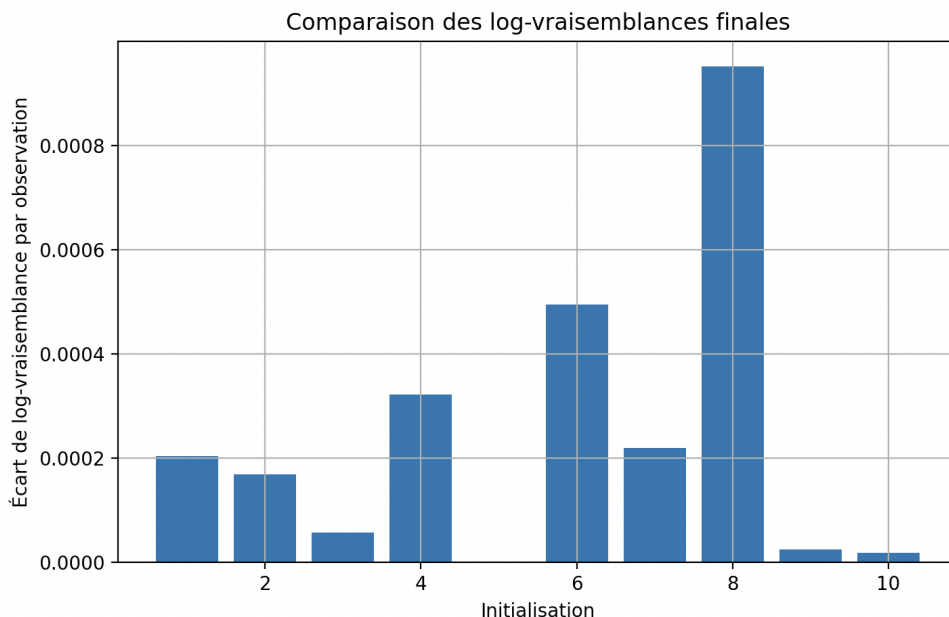


FIGURE 8 – Comparison of final log-likelihoods according to initialisation.

Figures 9 and 10 respectively show the stability of the transition matrix A and the emission matrix B . These two matrices exhibit larger differences depending on the initialisation. In particular,

initialisations 1, 2, 6, 7, and 8 lead to relatively large distances from the best solution. This shows that the estimated parameters are not perfectly stable.

This result is consistent with the expected behaviour of Baum–Welch : the algorithm may converge to different local maxima depending on the starting point. Thus, two initialisations may yield close final log-likelihoods while producing different matrices A and B . In other words, likelihood alone is not always sufficient to guarantee that the estimated regimes have exactly the same interpretation.

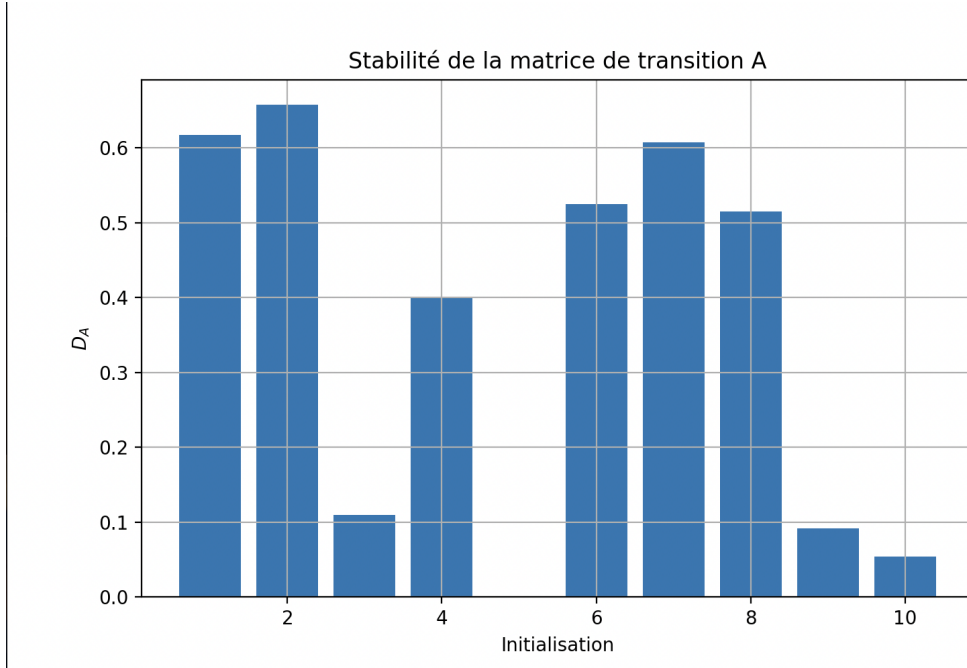


FIGURE 9 – Stability of the estimated transition matrix A .

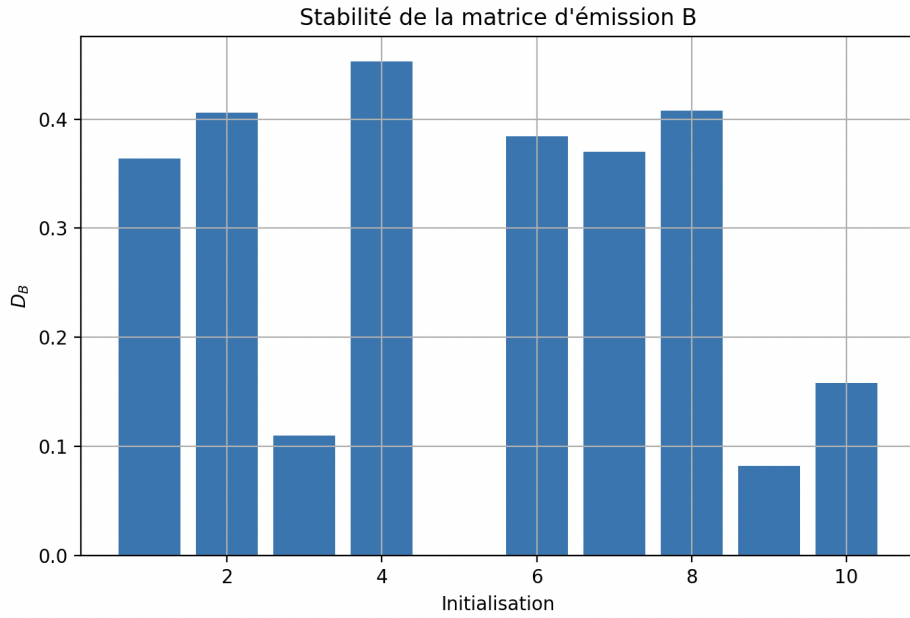


FIGURE 10 – Stability of the estimated emission matrix B .

Figure 11 makes it possible to interpret the stability of the transition matrix from a more concrete perspective by comparing average regime durations. We note that the initialisations for which the matrix A is far from the reference are also those for which the average regime durations vary most strongly. Initialisations 1, 2, 6, 7, and 8 thus exhibit large differences, whereas initialisations 3, 9, and 10 are much closer to the reference solution.

This means that some estimates do not merely describe different matrices : they also lead to a different interpretation of the persistence of the hidden regimes. For example, a regime may be estimated as more stable or more transient depending on the initialisation. This observation is important because, in a hidden Markov model, the average time spent in each state is essential information for interpreting the regimes.

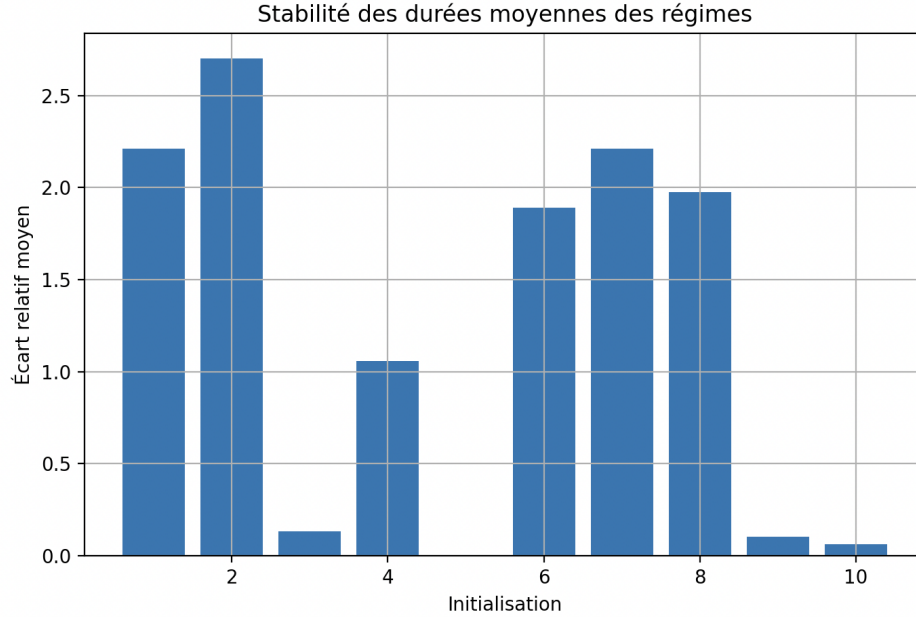


FIGURE 11 – Stability of average regime durations.

Overall, the results show that the algorithm succeeds in obtaining solutions that are close in terms of likelihood, but that the estimated parameters may vary substantially depending on the initialisation. This difference is particularly visible for the transition matrix A , the emission matrix B , and the average regime durations. This confirms the relevance of running Baum–Welch several times with different initialisations, and then retaining the solution with the best final log-likelihood.

In our case, initialisations 9 and 10 appear very close to the reference solution, both in terms of likelihood and estimated parameters. Conversely, some initialisations yield parameters that are further away despite a relatively close final likelihood. We may therefore conclude that the algorithm performs correctly on the simulated data, but that its result remains sensitive to initialisation, which is a standard limitation of EM-type methods.

5.4 Detection of Volatility Regimes on the VIX

In this section, we apply a hidden Markov model to the VIX, i.e. the implied volatility index of the S&P 500. The objective is to examine whether an HMM can automatically identify different market regimes from the evolution of volatility. Indeed, the VIX is often interpreted as an indicator of the level of stress or uncertainty in financial markets. A low VIX level generally corresponds to a calm period, whereas a high level is associated with periods of tension or crisis.

The data used are daily VIX observations between 2004 and 2024. After importing the data, we mainly retain the daily closing price of the VIX, denoted by V_t . In order to avoid constructing the model directly on the raw VIX level, we introduce several variables describing its dynamic

behaviour. We first consider the logarithm of the VIX :

$$X_t = \log(V_t),$$

then the daily log-return :

$$R_t = X_t - X_{t-1}.$$

We also compute two measures of realised volatility, obtained as rolling standard deviations of the log-returns :

$$\sigma_t^{(20)} = \text{std}(R_{t-19}, \dots, R_t),$$

and

$$\sigma_t^{(5)} = \text{std}(R_{t-4}, \dots, R_t).$$

The first measure provides an estimate of realised volatility over approximately one trading month, whereas the second captures shorter and more reactive dynamics.

We therefore use the following vector as observed variables

$$Y_t = \begin{pmatrix} R_t \\ \sigma_t^{(20)} \\ \sigma_t^{(5)} \end{pmatrix}.$$

These variables are then standardised before model estimation so that the different components have comparable orders of magnitude.

We consider a hidden Markov model with three hidden states :

$$S_t \in \{0, 1, 2\}.$$

Conditionally on hidden state $S_t = i$, the observation Y_t is assumed to follow a multivariate Gaussian distribution :

$$Y_t \mid S_t = i \sim \mathcal{N}(\mu_i, \Sigma_i),$$

where μ_i is the mean vector associated with regime i and Σ_i is its covariance matrix.

Model estimation is carried out using the Baum–Welch algorithm. Once the model has been estimated, the most likely sequence of hidden states is obtained using the Viterbi algorithm.

In our application, we selected three regimes in order to obtain a simple interpretation of market phases :

$$\text{calm regime,} \quad \text{intermediate regime,} \quad \text{crisis regime.}$$

Since the states obtained by the HMM have no natural ordering, we then reorder them according to the average realised volatility $\sigma_t^{(20)}$. The regime with the lowest average volatility is called the calm regime, the one with intermediate average volatility is called the intermediate regime, and the

one with the highest average volatility is called the crisis regime.

The average statistics obtained for the three regimes are as follows :

Regime	Average VIX	Average R_t	Average $\sigma_t^{(20)}$	Number of observations
Calm	16.3809	0.0007	0.0466	2364
Intermediate	20.0225	-0.0021	0.0719	1751
Crisis	23.6703	0.0018	0.1134	1003

The estimated transition matrix, after reordering the states, is given by :

$$\hat{A} = \begin{pmatrix} 0.972 & 0.020 & 0.007 \\ 0.038 & 0.941 & 0.021 \\ 0.000 & 0.053 & 0.947 \end{pmatrix}.$$

The diagonal coefficients are high. This means that the regimes are persistent.

The following figure represents the VIX over the period studied, with a background colour indicating the regime estimated by the HMM.

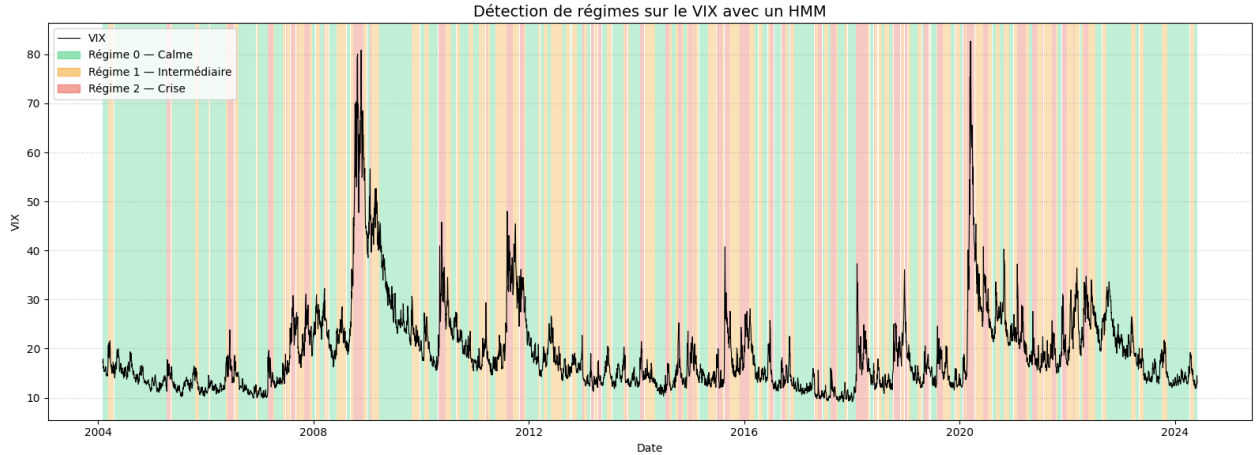


FIGURE 12 – Regime detection on the VIX index between 2004 and 2024

We observe that the model mainly identifies calm phases, associated with the lowest-volatility regime, while switching to the intermediate and crisis regimes during episodes of high instability. These changes appear in particular around major increases in the VIX, such as during the 2008 financial crisis, the sovereign debt crisis in 2011–2012, and the volatility shock observed in 2020. The model therefore appears capable of automatically identifying periods of market stress from the variables constructed using the VIX.

It should nevertheless be emphasised that the estimated regimes do not directly have an economic

interpretation. The HMM classifies observations according to their statistical properties, in particular returns and realised volatility levels, but it does not itself associate a hidden state with a specific economic event. The interpretation of the regimes must therefore be performed a posteriori by comparing the detected periods with known episodes of market tension.

The results also depend on the modelling choices, in particular the variables used, the number of hidden states, and the initialisation of the algorithm. The choice of three regimes provides a natural interpretation here, but other specifications could lead to a different segmentation.

This application thus illustrates the relevance of hidden Markov models for the analysis of financial series. In the case of the VIX, the HMM provides a simple probabilistic method for classifying market periods according to their volatility level and persistence dynamics.

5.5 Analysis of the Quality of the Baum–Welch Estimation on the VIX

We apply the Baum–Welch evaluation criteria to the HMM estimated on the VIX. The observations used are the same as previously :

$$Y_t = \begin{pmatrix} R_t \\ \sigma_t^{(20)} \\ \sigma_t^{(5)} \end{pmatrix},$$

where R_t is the daily log-return of the VIX, $\sigma_t^{(20)}$ is the realised volatility over twenty days, and $\sigma_t^{(5)}$ is the realised volatility over five days. The variables are standardised before estimation. The model has three hidden states and multivariate Gaussian emissions.

The series used contains 5118 observations, from 2 February 2004 to 31 May 2024.

Log-Likelihood Convergence

Baum–Welch is run with several initialisations. The best solution is obtained at the tenth initialisation. The final log-likelihood is

$$\ell(\hat{\lambda}) = -14300.087178,$$

corresponding to an average log-likelihood of

$$\frac{\ell(\hat{\lambda})}{T} = -2.794077.$$

Criterion	Value
Number of observations T	5118
Best initialisation	10
Number of iterations	36
Final log-likelihood	-14300.087178
Average log-likelihood	-2.794077
Final gain per observation	9.728857×10^{-11}
Final relative gain	3.481714×10^{-11}
Number of detected decreases	0

TABLE 7 – Convergence of Baum–Welch on the VIX.

The log-likelihood is increasing and no numerical decrease is detected. The final gain per observation is far below 10^{-6} , which indicates that the algorithm has stabilised. The convergence is therefore satisfactory.

Stability of the Estimated Parameters

Since Baum–Welch depends on the initialisation, the estimation is repeated ten times. The states are then aligned in order to correct for label switching. For the emissions, the discrete criterion D_B is replaced by a Gaussian distance based on a symmetrised Kullback–Leibler divergence between the emission distributions.

Stability criterion	Average value
D_π	$2.563015 \times 10^{-109}$
D_A	5.050970×10^{-7}
Gaussian D_B	7.307790×10^{-10}
Mean relative duration difference	1.257620×10^{-5}
Maximum log-likelihood difference per observation	1.587589×10^{-10}

TABLE 8 – Stability of the Estimates according to Initialisation.

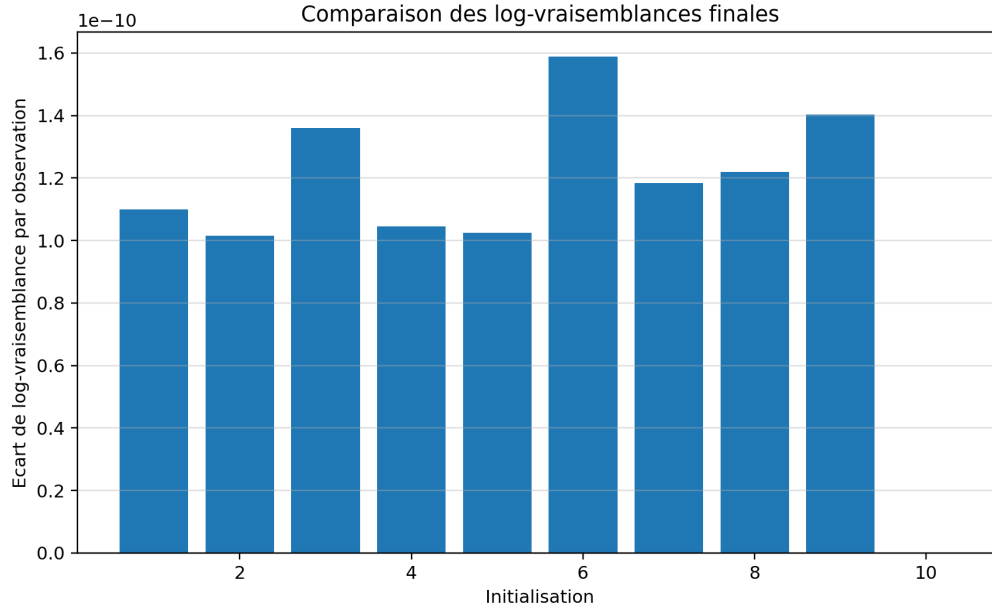


FIGURE 13 – Comparison of final log-likelihoods.

The differences between initialisations are extremely small. The final log-likelihoods are almost identical, the transition matrices are very close, and the Gaussian emissions remain stable after state alignment. Unlike the case in which Baum–Welch converges to several distinct local optima, the results obtained here indicate a very stable solution for this choice of variables and number of states.

Out-of-Sample Validation

The series is split chronologically : 4094 observations are used for training and 1024 observations are used for testing. The model is estimated on the training period and then evaluated on the subsequent period.

Segment	T	Log-likelihood	NLPD
Training	4094	-11536.441391	2.817890
Conditional test	1024	-2774.008272	2.708992
Independent test	1024	-2782.320895	2.717110

TABLE 9 – Out-of-sample validation of the HMM on the VIX.

The conditional test NLPD is 2.708992, compared with 2.817890 over the training period. Thus, the model does not exhibit out-of-sample deterioration on this chronological split. This comparison does not mean that the test period is always easier to predict, but it indicates that the parameters

estimated on the first part of the series retain a good ability to assign high density to future observations.

Conclusion

The criteria applied to the VIX provide a favourable evaluation of the Baum–Welch estimation. The log-likelihood converges without any numerical decrease, the different initialisations lead to virtually the same solution after state alignment, and the resulting regimes are interpretable in terms of market volatility. Out-of-sample validation confirms that the model remains stable over the test period. The main difference from the simulation is that there are no observable true hidden states here : the evaluation therefore relies on convergence, parameter stability, the economic consistency of the regimes, and predictive performance.

6 References

Références

- [1] Rabiner, L. R. (1989). *A tutorial on hidden Markov models and selected applications in speech recognition*. <https://www.cs.ubc.ca/~murphyk/Bayes/rabiner.pdf>
- [2] Jurafsky, D., & Martin, J. H. (2026). *Hidden Markov Models*. In *Speech and Language Processing* (Chapter A, draft of January 6, 2026). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/A.pdf>
- [3] Sublime, J. (2022). *Data Analysis – Lecture 9 : Time series analysis, Hidden Markov Models*. Engineering school lecture notes, France. HAL, hal-03845332. <https://hal.science/hal-03845332/document>
- [4] Baum, L. E., & Petrie, T. (1966). *Statistical inference for probabilistic functions of finite state Markov chains*. The Annals of Mathematical Statistics, 37(6), 1554–1563. <https://doi.org/10.1214/aoms/1177699147>
- [5] Baum, L. E., Petrie, T., Soules, G., & Weiss, N. (1970). *A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains*. The Annals of Mathematical Statistics, 41(1), 164–171. <https://doi.org/10.1214/aoms/1177697196>
- [6] Wu, C. F. J. (1983). *On the convergence properties of the EM algorithm*. The Annals of Statistics, 11(1), 95–103. <https://doi.org/10.1214/aos/1176346060>
- [7] Leroux, B. G. (1992). *Maximum-likelihood estimation for hidden Markov models*. Stochastic Processes and their Applications, 40(1), 127–143. [https://doi.org/10.1016/0304-4149\(92\)90141-C](https://doi.org/10.1016/0304-4149(92)90141-C)

- [8] Lember, J., & Koloydenko, A. A. (2014). *Bridging Viterbi and posterior decoding : A generalized risk approach to hidden path inference based on hidden Markov models*. Journal of Machine Learning Research, 15, 1–58. <https://jmlr.org/papers/v15/lemb14a.html>
- [9] Akaike, H. (1974). *A New Look at the Statistical Model Identification*. In E. Parzen, K. Tanabe, & G. Kitagawa (Eds.), *Selected Papers of Hirotugu Akaike* (pp. 215–222). Springer Series in Statistics. Springer, New York, NY. <https://doi.org/10.1109/TAC.1974.1100705>